

République Tunisienne
Ministère de
l'Enseignement
Supérieur
et de la Recherche
Scientifique



الجمهورية التونسية
وزارة التعليم العالي
والبحث العلمي

Compte rendu TP n°2

Application Mobile sur Android



Auteur

Karim Dhafer

Matière

Développement Mobile sur Android

Classe

L3-DSI3

Année universitaire

2024-2025

Sommaire

Introduction	1
Objectifs	1
1. Construction de l'interface	2
1.1 Manipulation des composants UI et leur arborescence	2
1.2 Manipulation de la composante ImageView	4
1.3 Gestion des attributs	4
1.4 Interface finale	6
2. Codage en Java sous Android suivant le MVC	6
2.1 Codage de la partie Modèle	7
2.2 Codage de la partie Controleur	7
2.3 Codage de la partie Vue	8
3. Réalisation des tests unitaires	11
3.2 Codage de la classe ProfilTest	12
4. Génération de la documentation avec JavaDoc	12
4.1 Écriture des commentaires du programme	12
4.2 Génération de la documentation	13
5. Application finale	14
Conclusion	15

Liste des figures

1	Palette et arborescence composants Android Studio	2
2	Emplacement LinearLayouts	2
3	Bouton de distribution de taille égale entre les composants	3
4	Arborescence finale des composants UI	3
5	Fentêtre de sélection des ressource	4
6	Champ d'attribut texte	5
7	Champ d'attribut texte	5
8	Champ d'attribut id	6
9	Interface finale	6
10	Création du test	11
11	Propriétés de création du test	11
12	Résultat du test	12
13	Fenêtre JavaDoc	13
14	Résultat de application finale	14

Introduction

Ce projet se concentre sur le développement d'une application mobile innovante qui permet de calculer l'IMG d'une personne fournissant une interface conviviale, réalisée avec Android Studio en utilisant le langage de programmation Java.

Cette application a été conçue dans le but de pratiquer le développement mobile sur la plateforme Android.

Objectifs

- Construction des interfaces.
- Codage en Java sous Android suivant le MVC.
- Réalisation des tests unitaires.
- Génération automatique de la documentation avec JavaDoc.
- Codage des actions événementielles.
- Exploite des ressources et des objets graphiques dans le code.
- Construction d'une application mobile Android simple et fonctionnel.

1. Construction de l'interface

Dans cette partie, on va apprendre comment construire l'interface graphique en utilisant les outils d'Android Studio et en manipulant le fichier **activity_main.xml**

1.1 Manipulation des composants UI et leur arborescence

- Double-cliquer sur les composants de la palette à utiliser sans les lâcher, puis les faire glisser vers le Component Tree sous le conteneur `ConstraintLayout`.

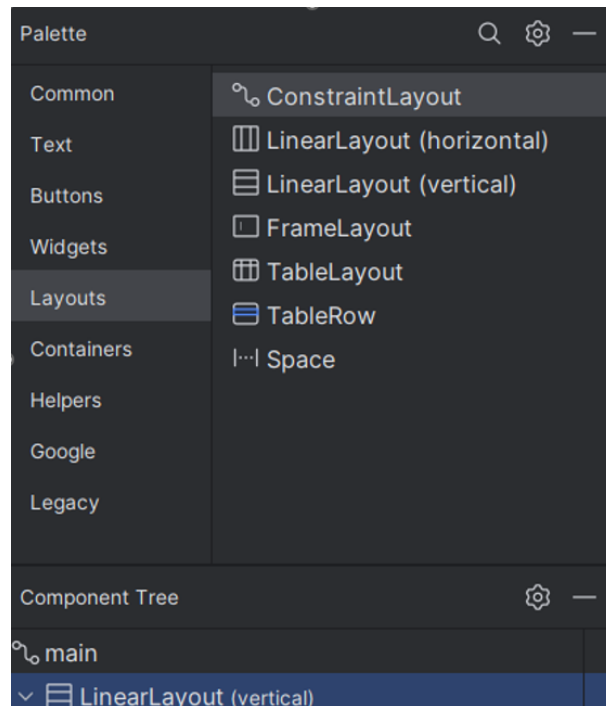


FIG. 1 : Palette et arborescence composants Android Studio

- On a intérêt à disposer 3 grands composants alignés horizontalement donc on ajoute une `LinearLayout Vertical` qui va les contenir. Puis en glissant ces trois-là, on les dépose sous le `LinearLayout Vertical`.

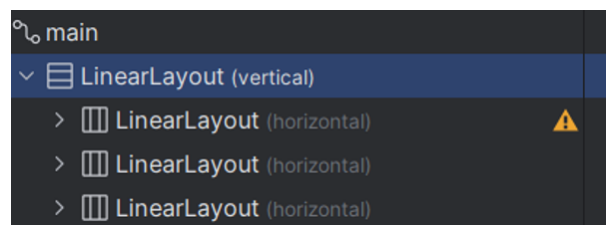


FIG. 2 : Emplacement LinearLayouts

- Ajuster les tailles des composants pour qu'elles soient égales en les sélectionnant tous en même temps, puis en cliquant sur ce bouton en dessous.

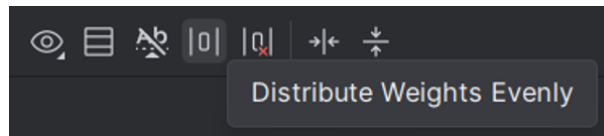


FIG. 3 : Bouton de distribution de taille égale entre les composants

- Appliquer tout ce qui est avant sur tous les autres composants ajouter, en faisant attention à mettre les composants désirés sous la composante mère propre afin d'avoir l'arborescence des composantes correcte.

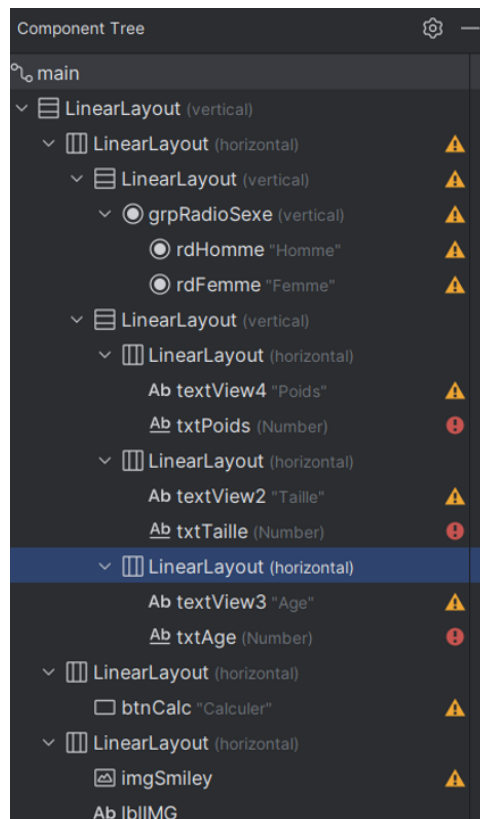


FIG. 4 : Arborescence finale des composants UI

Attention

- Il faut parcourir la section 2 (ci-dessous) avant d'intégrer la composante Image dans l'interface.
- Il ne faut pas déplacer une composante sous une composante mère inappropriée, car cela engendrera des problèmes d'emplacement des composantes dans l'interface, ça peut être corrigé en glissant la composante sous sa mère.

1.2 Manipulation de la composante ImageView

Dans cette partie, on va apprendre comment introduire des images dans l'interface de l'application.

1. Télécharger les images à partir de n'importe quelle source.
2. Ajouter ces images sous le dossier **drawable** pour qu'elles seront mise en considération comme des ressources dans l'application.
3. Ajouter la composante **ImageView**.
4. Ajouter l'image à partir du fenêtre qui a été ouverte.

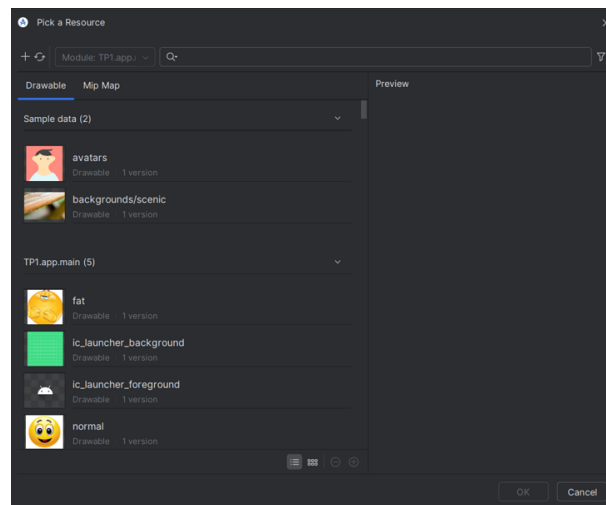


FIG. 5 : Fentêtre de sélection des ressource

1.3 Gestion des attributs

Dans cette partie, on a intérêt à savoir comment manipuler les attributs des composants pour qu'ils suivent la forme désiré.

- Changer les propriétés `gravity` et `layout_gravity` des composants qui sont responsables de la position du contenu des objets graphiques.
gravity : La position du contenu par rapport à l'élément lui-même.
layout_gravity : La position du contenu par rapport à l'élément parent.

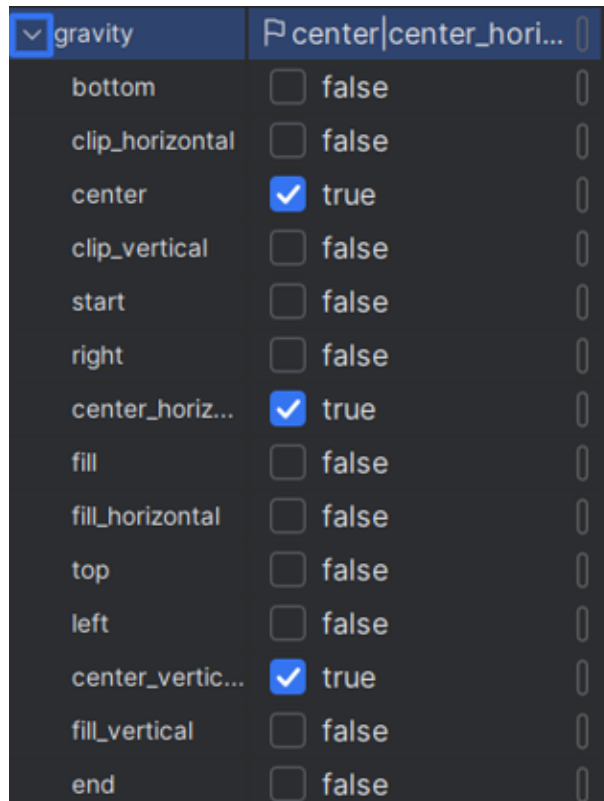


FIG. 6 : Champ d'attribut texte

- Pour afficher le texte désiré, on peut Éditer les textes à afficher sur ces composants (sauf l'image et les champs de nombre).

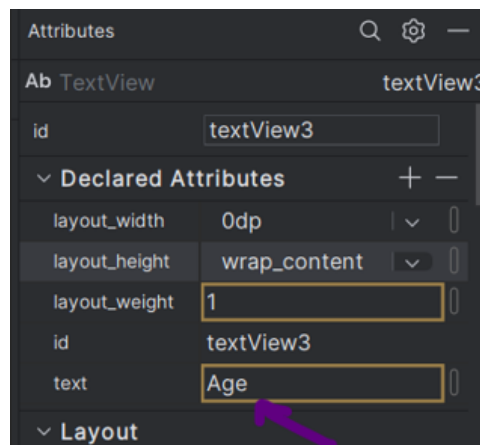


FIG. 7 : Champ d'attribut texte

- Pour exploiter les composants dans le code, on peut leur assigner des ID spécifiques et significatifs pour faciliter le codage et rendre le code plus lisible.

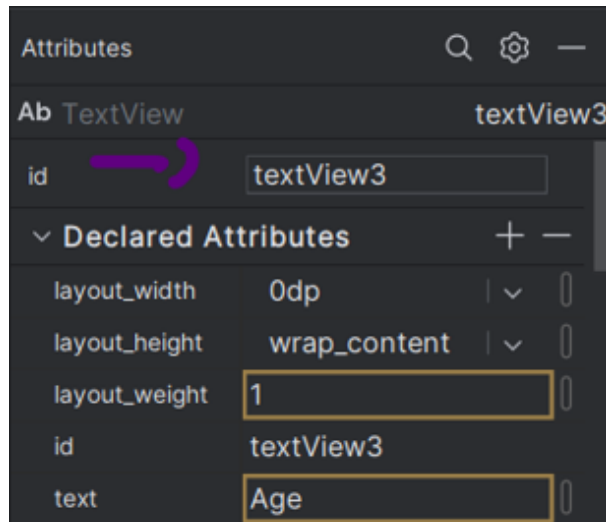


FIG. 8 : Champ d'attribut id

1.4 Interface finale

À ce stade, il faut avoir une interface similaire à celle au-dessous.

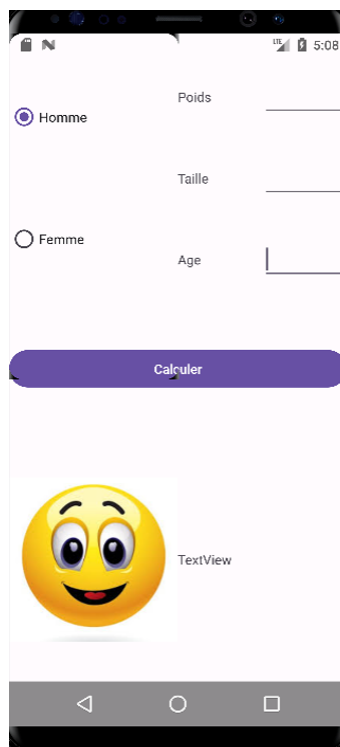


FIG. 9 : Interface finale

2. Codage en Java sous Android suivant le MVC

Tout d'abord, pour suivre les normes de l'architecture MVC, on crée 3 packages ; contrôleur, modèle et vue. Puis on transfère le fichier **MainActivity.java** dans le package vue

car il est le responsable à l’affichage de l’application. Maintenant on peut commencer le codage du partie modèle.

2.1 Codage de la partie Modèle

Dans cette partie, il faut créer la classe Java **Profil** qui est la classe contenant les attributs du profil, afin de calculer l’IMG et d’afficher l’image qui lui correspond.

```
1  public class Profil {
2      ... // Déclaration des constantes et des attributs de la classe.
3      public Profil(Integer poids, Integer taille, Integer age, Integer
         sexe) {
4          this.sexe = sexe;
5          this.age = age;
6          this.taille = taille;
7          this.poids = poids;
8          this.calculIMG();
9          this.resultIMG();
10     }
11     ... // Getters et Setters.
12     private void calculIMG() {
13         float tailleM = ((float)taille) / 100;
14         this.img = (float) ((1.2 * poids / (tailleM * tailleM)) +
            (0.23 * age) - (10.83 * sexe) - 5.4);
15     }
16
17     private void resultIMG() {
18         Integer min;
19         Integer max;
20         if (sexe == 0) { // femme
21             min = minFemme;
22             max = maxFemme;
23         } else {
24             min = minHomme;
25             max = maxHomme;
26         }
27
28         message = "normal";
29         if (img < min) {
30             message = "trop faible";
31         } else {
32             if (img > max) {
33                 message = "trop élevé";
34             }
35         }
36     }
```

2.2 Codage de la partie Controleur

Dans cette section, on doit créer la classe Java **Controle** sous le package **Controleur**. Cette classe joue un rôle central dans notre application en gérant l’interaction entre l’in-

terface utilisateur et la logique métier en suivant le modèle **Singleton**. Elle a d'objectif d'assurer que les données du profil sont correctement créées et accessibles à partir d'autres parties de l'application.

```
1      public final class Controle {
2          private static Controle instance = null;
3          private Profil profil;
4
5          private Controle() {
6              super();
7          }
8
9          public static final Controle getInstance() {
10             if (Controle.instance == null) {
11                 Controle.instance = new Controle();
12             }
13             return Controle.instance;
14         }
15
16         public void creerProfil(Integer poids, Integer taille,
17             Integer age, Integer sexe) {
18             profil = new Profil(poids, taille, age, sexe);
19         }
20
21         public float getImg() {
22             return profil.getImg();
23         }
24
25         public String getMessage() {
26             return profil.getMessage();
27         }
28     }
```

2.3 Codage de la partie Vue

Dans cette section, on doit coder la classe Java **MainActivity** pour gérer les actions événementielles de l'utilisateur.

- Au début, on doit réécrire la méthode **onCreate()** qui est la responsable d'afficher l'interface lors du lancement
- puis, pour afficher l'interface **activity__main.xml** dans l'écran, on doit utiliser la méthode **setContentView(R.layout.activity__main)** ;

```
1      protected void onCreate(Bundle savedInstanceState) {
2          super.onCreate(savedInstanceState);
3          EdgeToEdge.enable(this);
4          setContentView(R.layout.activity__main);
5          init();
6          this.controle = Controle.getInstance();
7          ecouteCalcul();
8      }
```

```

8         ViewCompat.setOnApplyWindowInsetsListener(
            findViewById(R.id.main), (v, insets) -> {
9             Insets systemBars = insets.getInsets(
                WindowInsetsCompat.Type.systemBars());
10            v.setPadding(systemBars.left, systemBars.top
                , systemBars.right, systemBars.bottom);
11            return insets;
12        });
13    }

```

- Maintenant, il faut initialiser les champs à manipuler par relier les variables Java avec les objets graphiques créés.

```

1        // Propriétés
2        private EditText txtPoids;
3        private EditText txtTaille;
4        private EditText txtAge;
5        private RadioButton rdHomme;
6        private TextView lblIMG;
7        private ImageView imgSmiley;
8        private Controle controle;
9
10       private void init() {
11           txtPoids = (EditText) findViewById(R.id.txtPoids
12           );
13           txtTaille = (EditText) findViewById(R.id.
14           txtTaille);
15           txtAge = (EditText) findViewById(R.id.txtAge);
16           rdHomme = (RadioButton) findViewById(R.id.
17           rdHomme);
18           lblIMG = (TextView) findViewById(R.id.lblIMG);
19           imgSmiley = (ImageView) findViewById(R.id.
20           imgSmiley);
21       }

```

- En fin, pour afficher le résultat après le clic sur le bouton calculer, il faut écrire un code qui représente un listener (un écouteur) qui écoute les événements de cliques sur le bouton et qui exécute la méthode **afficheResult()**.

```

1        private void ecouteCalcul() {
2            ((Button) findViewById(R.id.btnCalc)).
3            setOnClickListener(new Button.OnClickListener
4            () {
5                @Override
6                public void onClick(View v) {
7                    // Toast.makeText(MainActivity.this, "
8                    test", Toast.LENGTH_SHORT).show();
9                    // Log.d("message", "clic sur le btn
10                    *****");
11
12                    Integer poids=0;
13                    Integer taille=0;
14                    Integer age=0;

```

```

10         Integer sexe=0;
11         try {
12             poids = Integer.parseInt(txtPoids.
13                 getText().toString());
14             taille = Integer.parseInt(txtTaille.
15                 getText().toString());
16             age = Integer.parseInt(txtAge.
17                 getText().toString());
18         } catch (Exception e) {};
19         if (rdHomme.isChecked()) {
20             sexe = 1;
21         }
22         //contrôle des données saisies
23         if (poids == 0 || taille == 0 || age ==
24             0) {
25             Toast.makeText(MainActivity.this, "
26                 Saisie incorrecte", Toast.
27                     LENGTH_SHORT).show();
28         } else {
29             afficheResult(poids, taille, age,
30                 sexe);
31         }
32     }
33 }
34
35 private void afficheResult(Integer poids, Integer
36     taille, Integer age, Integer sexe) {
37     // Création du profil et récupération des infos
38     this.controle.creerProfil(poids, taille, age,
39         sexe);
40     float img = this.controle.getImg();
41     String message = this.controle.getMessage();
42
43     // affichage
44     if (message == "normal") {
45         imgSmiley.setImageResource(R.drawable.normal
46             );
47         lblIMG.setTextColor(Color.GREEN);
48     } else {
49         lblIMG.setTextColor(Color.RED);
50         if (message == "trop faible") {
51             imgSmiley.setImageResource(R.drawable.
52                 skinny);
53         } else {
54             imgSmiley.setImageResource(R.drawable.
55                 fat);
56         }
57     }
58     lblIMG.setText(String.format("%.01f", img) + " :
59         IMG " + message);

```

3. Réalisation des tests unitaires

Dans cette partie, on va apprendre comment tester les fonctions principales dans le code en utilisant **JUnit** qui est un framework conçue pour faciliter le testage des codes écrites en Java. Ces tests sont appelés des **tests unitaires**.

3.1 Initialisation de la classe test

1. Se positionner sur la classe à tester.
2. Cliquer sur **CTRL+SHIFT+T** ou parcourir à partir de la barre d'outil **Navigate > Test**.
3. Cliquer sur **Create new test**.

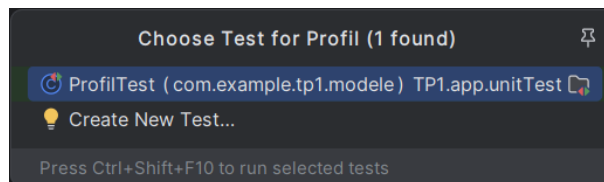


FIG. 10 : Création du test

4. Choisir la librairie voulue, puis sélectionner les méthodes à tester en laissant le nom intacte (c'est une convention de nommage que le nom de la classe test sera sur la forme Class_name suivi du mot Test), enfin cliquer sur créer.

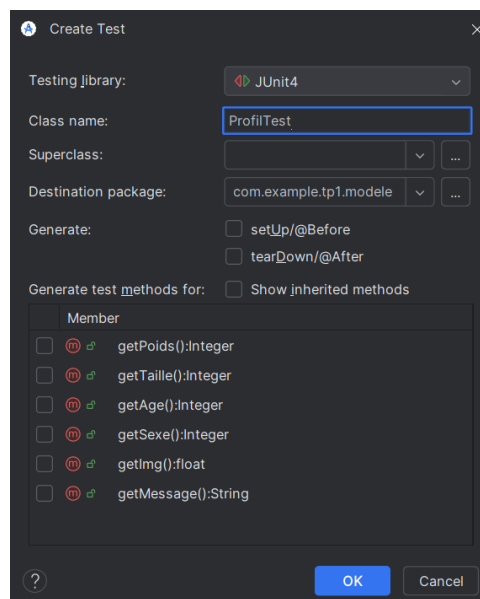


FIG. 11 : Propriétés de création du test

5. Ne rien changer dans le chemin car c'est automatiquement bien positionné.

3.2 Codage de la classe ProfilTest

Afin de tester les méthodes de la classe Profil, on doit initialiser des variables conçues par les tests comme le profil à tester, son résultat et image attendues. Puis en utilisant les méthodes prédéfinies et les annotations du Framework, on peut les tester correctement”.

@Test : annotation pour dire que la méthode ci-dessous est un test unitaire.

assertEquals : méthode prédéfinie pour comparer le résultat attendu avec le résultat actuel retourné par la méthode de la classe Profil.

```
1  public class ProfilTest {
2      // Création du profil avec ses résultats correctes
3      private Profil profil = new Profil(67, 165, 35, 0);
4      private float img = (float) 32.2;
5      private String message = "trop élevé";
6
7      @Test
8      public void getImg() {
9          assertEquals(img, profil.getImg(), (float) 0.1);
10     }
11
12     @Test
13     public void getMessage() {
14         assertEquals(message, profil.getMessage());
15     }
16 }
```

Finalement, le résultat doit apparaître comme ça pour un test validé.

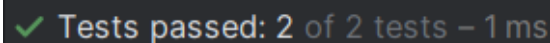


FIG. 12 : Résultat du test

4. Génération de la documentation avec Javadoc

Dans cette partie, on va apprendre comment générer automatiquement la documentation de notre application Java en utilisant **Javadoc**, un framework Populaire dédié à la documentation des applications Java.

4.1 Écriture des commentaires du programme

Au début, il faut commenter les codes Java pour que le framework génère une documentation adéquate, prenant l'exemple de la méthode **creerProfil**.

```
1  /**
2   * Création du profil
3   * @param poids
4   * @param taille en cm
5   * @param age
```

```

6      * @param sexe 1 pour homme et 0 pour femme
7      */
8      public void creerProfil(Integer poids, Integer taille, Integer
          age, Integer sexe) {
9          profil = new Profil(poids, taille, age, sexe);
10     }

```

4.2 Génération de la documentation

1. Ajouter un dossier **nom_de_votre_appDoc** dans le même dossier contenant l'application.
2. Cliquer sur le package **com.example.nom_de_votre_app**.
3. Naviger vers la barre d'outil **Tools > Generate JavaDoc**.
4. Configurer les propriétés de la fenêtre JavaDoc en changeant le chemin au dossier créé.

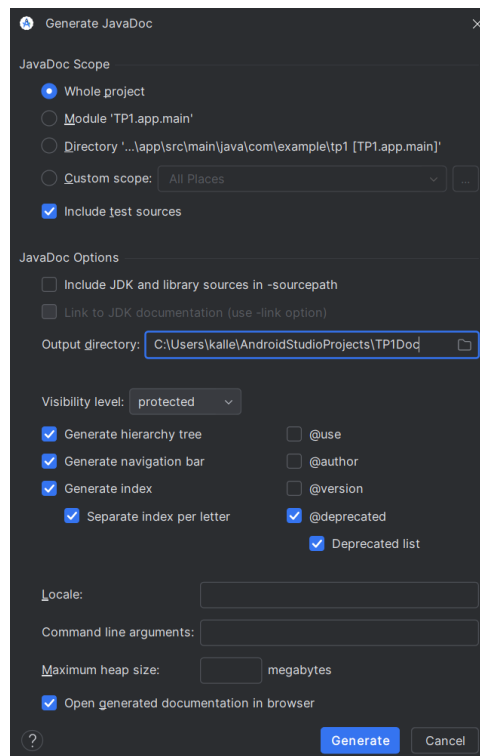


FIG. 13 : Fenêtre JavaDoc

Le résultat sera :

Des fichiers HTML permettant de naviguer entre la documentation des fonctionnalités de notre application.

Intérêt

Le vrai intérêt de cette documentation est de permettre aux autres de comprendre notre application sans avoir besoin de consulter le code.

5. Application finale

Dernier mais non des moindres, après avoir compris ces notions et les avoir pratiquées dans le monde réel, nous aurons cette application qui calcule l'IMG fournissant une interface mobile conviviale.

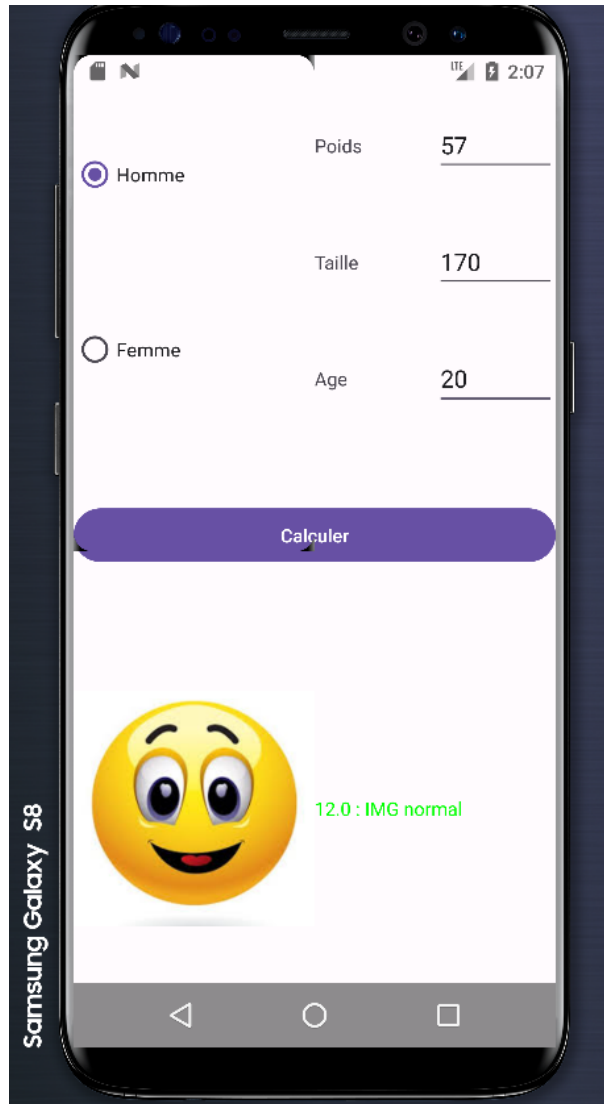


FIG. 14 : Résultat de application finale

Conclusion

En somme, ce rapport a permis de présenter les différentes étapes de développement et de mise en œuvre de notre application mobile. On a exploré les outils de **Android Studio**, l'architecture du projet, les différents frameworks à utiliser pour améliorer le projet.